



Elektronski fakultet u Nišu

DCASE 2024 Zadatak 6

Automatsko opisivanje audio-sadržaja

Student: Danilo Milošević

Profesor: Dejan Ćirić

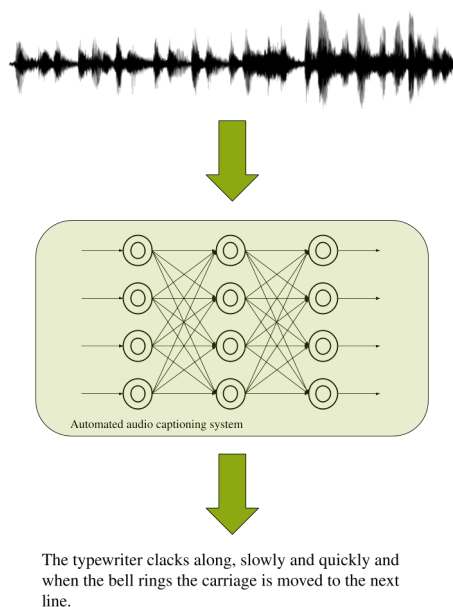
Sadržaj

1. Uvod	3
1.1 Definicija problema	3
1.2 Skup podataka za evaluaciju	3
1.3 Evaluacija i baseline performanse	4
1.3 Pokretanje modela	6
2. Opis baseline rešenja	6
2.1 Arhitektura modela	6
2.1.1 ConvNeXt enkoder	6
2.1.2 Projekcija	7
2.1.3 Dekoder	7
2.2 Trening in inferencija	8
3. Potencijalna unapredenja	9
3.1 JEPA arhitektura	9
3.2 Treniranje sa auxiliary loss	9
3.3 Potencijalni nedostaci	10

1. Uvod

1.1 Definicija problema

Automatsko opisivanje audio-sadržaja je problem generalnog opisivanja audio sadržaja tekстом. To je inter-modalni zadatak prevođenja, gde cilj nije vršiti direktno speech-to-text tj odrediti šta je tačno rečeno na snimku, već je cilj odrediti deskripciju zvučnog signala. Na primer kod razgovora dvoje ljudi, cilj nije odrediti šta je tačno rečeno već dati opis nalik “dvoje ljudi pričaju u maloj prostoriji”. Po definiciji problema sa DCASE 2024, model treba biti u stanju da prepozna kako i generalne koncepte (npr. prigušen zvuk) tako i fizičke osobine okoline (zvuk velikog automobila) kao i high-level koncepte (sat je zvonio tri puta).



Slika 1: Automatsko opisivanje audio-sadržaja

Po opisu problema, dozvoljeno je takmičarima da koriste eksterne skupove podataka koji su prigladni za dat problem kao i druge pre-trained modele kao što su Word2Vec, BER itd.

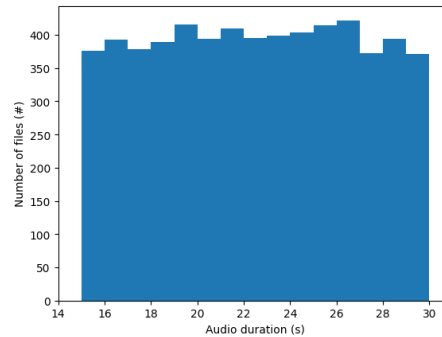
1.2 Skup podataka za evaluaciju

Za ovaj zadatak autori su za evaluaciju performanse modela izabrali Clotho V2 skup podataka, koji predstavlja proširenje originalnog skupa podataka Clotho v1 i sadrži 6972 audio zapisa trajanja od 15 do 30 sekundi, pri čemu je svaki zapis opisan sa pet tekstualnih opisa dužine od 8 do 20 reči. Ukupno obuhvata 34860 opisa. Skup podataka je kreiran od audio zapisa sa platforme Freesound, dok su opisi prikupljeni putem platforme Amazon Mechanical Turk od govornika engleskog jezika. Tokom obrade uklonjeni su vlastita imena, jedinstvene reči i transkripti govora kako bi se naglasak stavio na opis zvučnog sadržaja.

Clotho v2 sadrži oko 4.500 različitih reči i podeljen je na četiri dela: razvojni (development), validacioni (validation), evaluacioni (evaluation) i test (testing) skup.

Audio zapisi traju od 10 do 30s, dobrog kvaliteta (44.1 kHz i 16b). Sadržaj snimaka dosta varira, od zvukova

u prirodi (tok reke preko kamenja, zvuk životinja), ljudskog govora do zvuk rada mašina i različitih uređaja. Raspodela trajanja je data na sledećoj slici



Slika 2: Raspodela trajanja snimaka u Clotho skupu podataka

Clotho V2 je podeljene na sledeći način

- Razvojni podskup - sa 3839 klipova i 19195 opisa
- Validacioni podskup - sa 1045 klipova i 5225 opisa
- Evaluacioni skup - sa 1045 klipova i 5225 opisa
- Test skup - sa 1043 klipova i 5215 opisa.

Prva 3 podskupa možemo koristiti za trening modela dok je test skup za konačnu evaluaciju.

1.3 Evaluacija i baseline performanse

Za merenje performansi modela koriste se 7 metrike

1. **METEOR**

Poredi generisani opis sa referentnim tekstovima na nivou reči, uzimajući u obzir tačna poklapanja, sinonime, korene reči i parafraze. Zatim računa harmonijsku sredinu precision i recall uz penal za pogrešan redosled reči. Računa se kao

$$F_{mean} = \frac{10PR}{R + 9P}$$

stavljajući više fokusa na recall nego na precision. Zatim se računa kazna

$$Pen = 0.5 \cdot \left(\frac{m}{u}\right)^3$$

kao odnos m poklopljenih delova teksta i u ukupno poklopljenih reči. Konačni score je

$$Score = F_{mean} \cdot (1 - Pen)$$

2. **CIDEr**

Za razliku METEOR metrike, CIDEr meri koliko generisan opis (CIDEr je prvenstveno pravljen za opis slika) i pravi opis imaju slično značenje i gramatiku. Radi u par koraka

- U originalnom i referentnom zapisu sve reči mapira na njihove korene (npr fishing u fish).
- Svaka rečenica se zatim predstavlja kao skup n-grama prisutnih u rečenici (n-gram je skup od par reči, kod ove metrike 1 do 4 reči)

- Mera preklapanja treba da enkodira koliko često se n-gram u generisanoj rečenici javlja u referentnoj rečenici. Pored toga, n-gram koji nije prisutan u generisanoj rečenici ne bi trebao da se javi u referentnoj. Česti n-grami dobijaju manju težinu (kao što je the u engleskom jeziku). Za enkodiranje ovoga se koristi TF-IDF težine.
- Na osnovu ove težine dobijamo CIDER score korišćenjem TF-IDF težina za svaku kombinaciju izvornih i generisanih opisa.

3. **SPICE**

Umesto poređenja reči ili n-grama, ovde se parsira generisani i referentni tekst u tzv *scene* grafove i zatim se grafovi porede. Ova metrika bolje opisuje semantički sadržaj opisa i manje meri tačno poklapanje opisa.

4. **SPIDEr**

Nastao kao kombinacija *SPICE* i *CIDEr*, gde je cilj bio da se iskoristi prednost *CIDEr*-a (dobra korelacija sa ljudskom procenom tačnosti fraza) i *SPICE*-a koji dobro hvata sadržaj opisa.

5. **SPIDEr-FL** (*SPIDEr* with Fluency Error detection)

Modifikuje *SPIDEr* tako što prvenstveno proverava da li generisani opis ima ozbiljne greške, kao što su ponavljanje reči ili gramatičke greške, najčešće pomoću posebnog klasifikatora. Ako se detektuje greška opis dobija skor 0.

6. **FENSE**

Koristi S-BERT da izmeri semantičku sličnost između generisanih i referentnih opisa (kosinusna sličnost embedding vektora) a onda dodaje i detektor grešaka, slično kao kod prethodne metrike.

7. **Vocabulary**

Meri raznovrsnost izlaza modela, obično kao broj jedinstvenih reči ili n-grama, kako bi se detektovalo ako model uvek koristi iste fraze.

U DCASE2024, problemu 6, korišćen je FENSE za rangiranje metoda. Baseline arhitektura (koju ćemo obrađiti u sledećem poglavlju) postiže sledeće rezultate

Metric	Value
METEOR	0.1897
CIDEr	0.4619
SPICE	0.1335
SPIDEr	0.2977
SPIDEr-FL	0.2962
FENSE	0.5040
Vocabulary	551

1.3 Pokretanje modela

Samo pokretanje modela je bilo relativno prosto. Nakon povlačenja repozitorijuma sa GitHub-a je potrebno

1. **Kreirati virtuelni environment**, najbolje u Python 3.11 s obzirom na pakete koje treba instalirati.
2. **Instalirati pakete**
Ovde je najviše problema nastalo. Ako se projekat pokreće na CUDA grafičkama, instalacija je jednostavna. Kod AMD kartica može se koristiti ROCm, gde je podrška najbolja na Linuxu. Na Windows-u je na kraju najlakše bilo izbaciti DeepSpeed paket koji je svakako za trening i inferenciju na distribuiranim i multi-gpu uređajima kao i podesiti torch na CPU verziju, što je za inferenciju sasvim prihvatljivo.
3. **Pokretanje modela**
U okviru repozitorijuma je dato uputstvo kako pokrenuti model, međutim dat primer ne funkcioniše na Windowsu. Primer pokretanja se može naći u `test.py` skripti.

2. Opis baseline rešenja

2.1 Arhitektura modela

Baseline model se zove CNext-trans i to je klasična encoder-decoder arhitektura gde konvolucioni enkoder izvlači audio reprezentacije, a transformer dekode ih koristi da generiše tekstualni opis reč po reč. Analiziramo svaku od komponenta posebno

2.1.1 ConvNeXt enkoder

Razvoj ConvNeXt mreža je bio inspirisan uspesima vizuelnih transformera, gde je ideja bila adaptirati ideje koje su učinile transformere uspešnim na konvolucione mreže. Autori su u radu *A ConvNet for the 2020s* uzeli ResNet-50 kao osnovu a zatim postepeno koristili tehnike transformera kako bi poboljšali performanse. Ovo je učinjeno u više koraka.

1. Trening protokol

Pre bilo kakvih arhitekturnih promena autori su prvo primenili modernije tehnike treniranja i to konkretno

- Koristili su AdamW optimizator umesto SGD
- Povećali broj epoha
- Koristili nove augmentacione tehnike kao što su mixup i cutmix
- Label smoothing, gde se ciljna labela "uglađuje", npr ako je ciljna labela bio vektor $[0, 1, 0, 0]$ on se transformise u $[0.1, 0.7, 0.1, 0.1]$.
- Stochastic depth - slično kao dropout, međutim umesto isključivanja pojedinih neurona, ova tehnika preskače pojedine strukturne blokove.

2. Promene u strukturi blokova

- Kod ResNet-50 se koristila takva arhitektura gde se na slici vrši standardna konvolucija (nad svim kanalima istovremeno) sa veličinom kernela 3×3 . Kako je skupo primeniti 3×3 konvoluciju na svim kanalima, tehnika koju je ResNet koristio je da prvo se vrši 1×1 konvolucija, čime se veličina kanala zadržava a broj kanala smanjuje. To je onda praćeno 3×3 konvolucijom koja je sada jeftinija a zatim još jedna 1×1 konvolucija se vrši koja vraća broj kanala. Ovim model ne može da vidi dalje zavisnosti u podacima (s obzirom da je kernel mali), za razliku od transformera koji uče zavisnost svakog podatka od svih ostalih u ulazu.
Zbog ovoga je prva promena promeniti tip i redosled konvolucije. Prvo će se vršiti 7×7 dubinska

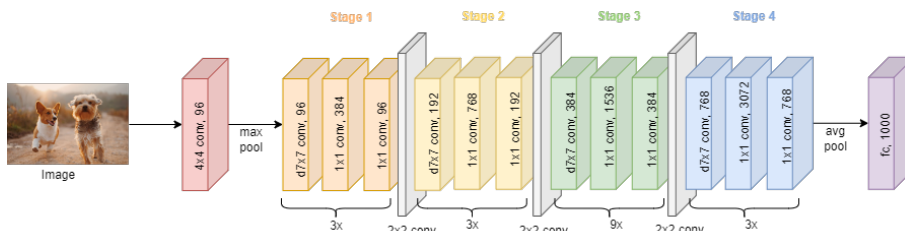
konvolucija (svaki kanal se posebno konvolira), zatim se radi 1x1 konvolucija sa većim brojem kanala, nakon čega sledi GELU aktivacioni sloj i na kraju 1x1 konvolucija koja smanjuje broj kanala. Ovim sada model prvo uči prostorne zavisnosti i to sa većim filterom (perceptivnim poljem), zatim povećava dimenzionalnost kako bi mogao kompleksnije nelinearne transformacije da vrši a zatim vraća broj kanala. Ovakva metoda je sličnija transformerima - prvo se vrši prostorna transformacija a zatim po kanalima.

- Druga promena, spomenuta gore je korišćenje GELU umesto ReLu kao i korišćenje aktivacije samo jednom u bloku umesto posle svake konvolucije. Ovo čini model opet sličnijim transformerima, koji imaju samo jedan nelinearan sloj (feed-forward network nakon attentiona koji je linearan)
- Poslednja promena u okviru blokova je normalizacija. ResNet je koristio BatchNorm nakon svake konvolucije, što se loše ponaša kod manjih batcheva i fine-tuninga. Umesto toga ConvNext koristi jedan LayerNorm po bloku, slično transformeru.

3. Promene u rasporedu blokova

- Kod SwinTransformera je korišćena raspored blokova 1:1:3:1, tj najveća računica je vršena u kasnijim slojevima, dok je ResNet koristio 3,4,6,3. Zbog ovoga ConvNeXt koristi raspored blokova (3,3,9,3).
- Prvi sloj mreže kod ResNet koristi 7x7 konvoluciju sa stride 2 i maxpool, kako bi smanjio dimenzionalnost ulaza. Sa druge strane, transformeri dele sliku na patcheve i formiraju embeddinge. Zato ConvNeXt prvo radi 4x4 konvoluciju sa stride 4, efektivno deleći ulaz na 4x4 delove.
- Umesto smanjivanja u prvom sloju sa stride 2, sada je odvojen LayerNorm sloj sa 2x2 konvolucijom kako bi se kombinovali patch-evi.

Kombinovanjem ovih tehnika je kreiran ConvNeXt, koji postiže praktično jednake rezultate sa vision transformerom, pri čemu koristi manje memorije, ima veći throughput i manje potrebnih FLOP-ova za inferenciju.



Slika 3: ConvNext

2.1.2 Projekcija

Sledeći sloj je jednostavan - kako ConvNeXt daje na izlazu 768-dimenzione vektore a dekodier u sledećem koraku koristi ulaz sa 256 dimenzija, dodat je projekcioni sloj - Dropout(0.5) - Linear (768 -> 256) - ReLu - Dropout(0.5)

2.1.3 Dekoder

Standardni PyTorch TransformerDecoder sa parametrima - Veličina ulaza: 256 - Broj attention head-ova: 8 - Broj dekodier slojeva: 6. - Dimenzija feedforward mreže: 2048. - GELU - Dropout: 0.2

Tok kroz dekodier je sledeći

1. Na ulaz dekodera se dovode audio frame embeddings iz projekcionog sloja kao i izlazi embedding sloja koji pretvara trenutno generisane tekst tokene u vektore, uz dodato poziciono kodiranje.
2. Unutar svakog od 6 slojeva dekodera, prvo se primenjuje self-attention nad tekst tokenima
3. Zatim se primenjuje cross-attention, gde svaki tekst token određuje koliko je važan svaki audio frame embedding za njegovu trenutnu reprezentaciju.
4. Poslednja tekst reprezentacija prolazi kroz linearni klasifikator verovatnoće za sledeću reč nad celim rečnikom.

2.2 Trening in inferencija

Kod treniranja modela su autori koristili sledeće tehnike za bolju generalizaciju

- Teacher forcing - tehnika kojom se izbegava poznat problem kod autoregresivnih modela. Ovaj tip modela predviđa sledeću reč na osnovu prethodnih, tako da svaka loše predviđena reč tokom treninga gomila grešku. Umesto toga se kao predhodno predviđena reč uvek daje tačna reč.
- Label smoothing, tehnika kojom se koriguje ciljni vektor koji model treba da predvidi. Ako su ciljne verovatnoće date sa 0 i 1, model se forsira da bude ekstremno samouveren, što dovodi do overfittinga.
- Mixup augmentacija, gde se model trenira nad instancama koje se mešaju sa drugim instancama iz batch-a. Ovakva augmentacija se primenjuje na audio ali i na tekstu. Na ovaj način model uči ne samo da prepozna tačne primere već i njihove interpolacije.
- AdamW optimizator, sa $5 \cdot 10^{-4}$ stopom učenja, weigh decay 2.0 i kosinusnim decay scheduler-om koji glatko smanjuje learning rate od inicijalne vrednosti do 0 u toku 400 epohe.
- Batch size 64 i gradient accumulation 8, s obzirom da treniranje sa 512 batch size bi zahtevao previše GPU memorije. Umesto toga se vrši trening sa batch-evima od 64 podataka, pri čemu se vrši trening nad 8 batch-a bez menjanja težina. Nakon svih 8 batch-a se akumuliraju gradijenti i menjaju se težine.

Tokom inferencije se koriste sledeća poboljšanja

- Beam search sa veličinom 3
Tokom klasične inferencije model bira jednu najverovatniju sledeću reč i nastavlja generisanje. Tu je problem što lokalno najbolji izbor sledeće reči ne vodi nužno do globalno najboljeg rezultat. Zato umesto toga model ide u n (ovde 3) najbolja smera, tj bira 3 najbolje sledeće reči paralelno, zatim odredi 3 sledeće reči za svaku od 3 izabranih reči i od tih kombinacija bira one sa najboljom kumulativnom verovatnoćom. Na ovaj način se mnogo više potencijalnih opisa istražuje.
- Zabrana ponavljanja bitnih reči
Tranformer dekoderi često ispoljavaju ponavljanje reči (npr dog is barking and dog is barking), tako da ova tehnika izbegava taj problem podešavanjem verovatnoće na 0 bitnim rečima (imenica, glagol, pridev) koje su već generisane.
- Ograničenje dužine, kojom želimo da izbegnemo da model generiše prekratke ili predugačke opise.

3. Potencijalna unapređenja

Ideja za potencijalno unapređenje rešenja je koristiti JEPA model kako bi dobili bolje latentne reprezentacije ulaznih audio signala kao i trenirati dekodera sa dva cilja - bolje predvideti latentne reprezentacije tekstualnog opisa kao i generisati opis.

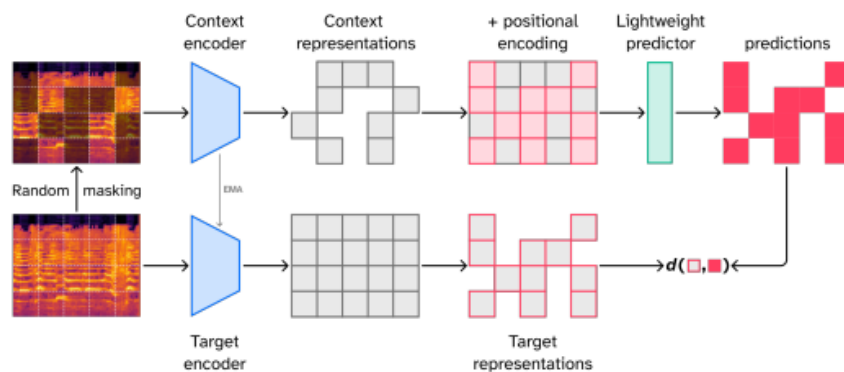
3.1 JEPA arhitektura

Joint-Embedding Predictive Architecture ili JEPA je self-supervised learning arhitektura kojom se vrši predviđanje latentne reprezentacije maskiranih delova ulaza. Kod slika JEPA je korišćena za generisanje kvalitetnijih latentnih reprezentacija slika, tj embedding vektora koji bi bolje očuvali značenje odnosno kontekst slike. U opštem slučaju JEPA funkcioniše na sledeći način

1. Deo ulaza (slika, audio) se maskira, tj sakrije.
2. Maskirani ulaz se dovodi na context encoder koji nemaskirani deo procesira i generiše njegovu reprezentaciju.
3. Target encoder uzima originalni, nemaskirani ulaz i generiše njegovu reprezentaciju.
4. Imamo manji prediktor koji koristi reprezentaciju dostupnog dela ulaza kako bi predvideo latentnu reprezentaciju maskiranog dela koju generiše target encoder.
5. Cilj je da se originalna i rekonstruisana latentna reprezentacija poklope

U više radova pokazano je da ovakav način self-supervised učenja može dovesti do kvalitetnijih latentnih reprezentacija koje se uspešno koriste u različitim downstream zadacima.

Audio JEPA može naučiti reprezentacije koje bolje opisuju semantički sadržaj audio signala, nezavisno od konkretnog zadatka. Takve reprezentacije mogu predstavljati kvalitetniji ulaz dekodera, naročito kada je količina označenih podataka ograničena. Koristićemo pre-trenirani Audio JEPA model i dodatno ga trenirati na Clotho skupu.



Slika 4: Audio JEPA

3.2 Treniranje sa auxiliary loss

S obzirom da se dekodera trenira sa ciljem da što bolje predvidi sledeću reč u opisu na osnovu ciljnog opisa, model se podstiče da reprodukuje tačan niz tokena iz referentnog opisa. Na primer ukoliko je ciljni opis “glasen zvuk automobila”, validan je i opis “bucan zvuk vozila/kola”, koje bi model smatrao pogrešnim. Zato je potencijalno unapređenje trenirati model sa dva cilja - Standardni cross-entropy gubitak koji sada koristi

baseline model. - Pored standardnog cross-entropy gubitka, može se uvesti dodatni semantički gubitak kojim se minimizuje rastojanje između embeddinga generisanog opisa i embeddinga referentnog opisa dobijenog pomoću prethodno istreniranog tekstualnog enkodera kao što je SBERT.

3.3 Potencijalni nedostaci

Dodavanjem Audio JEPA se povećava računaska složenost sistema i zahteva ili dodati fazu pretreniranja ili koristiti već treniran model. Pored toga, dodavanje semantičkog gubitka zahteva pažljivo izabrati težine ova dva gubitka, koje se potencijalno mogu i učiti.